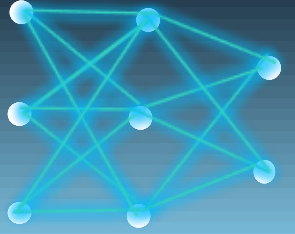




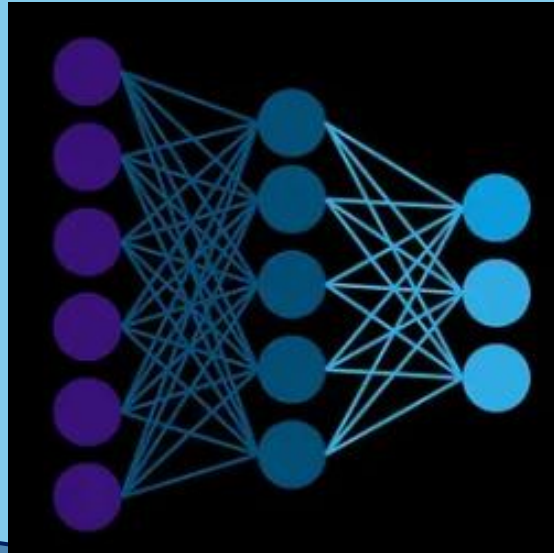
Deep Learning

Dr. Shay Maymon

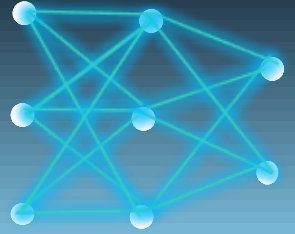
Architecture Design



$$\mathbf{h}^{(l)} = g^{(l)} \left(W^{(l)T} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)} \right), l = 1, 2, \dots, L$$

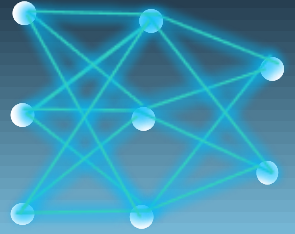


Architecture Design

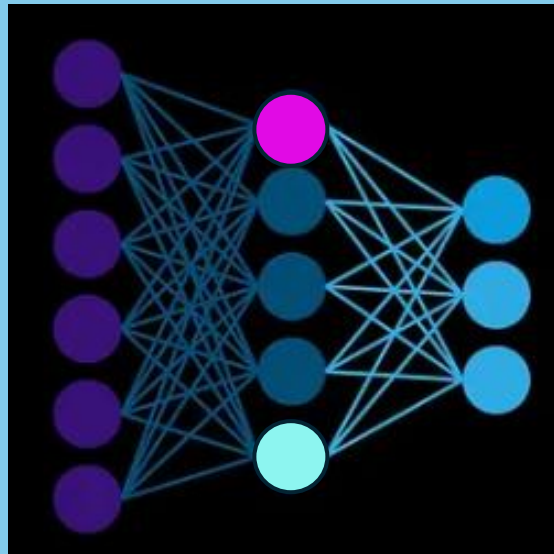


- Many architectures build a main chain but then add extra architectural features to it, such as **skip connections** going from layer i to layer $i + 2$ or higher
- Architecture design also determines layer connectivity: while standard neural networks use fully connected layers (every input connected to every output), many specialized architectures use **sparse connections**, where each input unit connects only to a limited subset of output units

Weight-space Symmetries



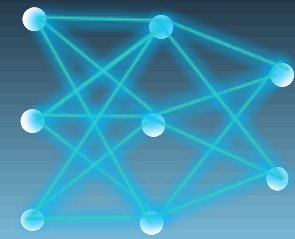
- Neural networks exhibit *weight space symmetry*: hidden units can often be **permuted** without changing the model's output, leading to many equivalent parameter configurations

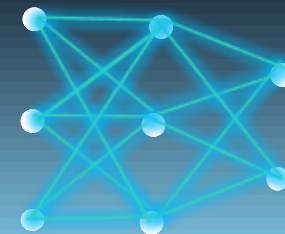


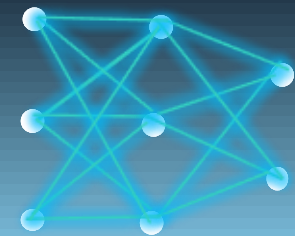
```

[[ 52  55  61  66  70  61  59  60  63  65  66  67  70 ... 89 91 93]
 [ 53  56  63  67  69  62  60  61  64  66  67  68  71 ... 90 92 94]
 [ 56  60  64  68  71  63  61  63  65  68  69  70  72 ... 91 93 95]
 [ 58  61  65  70  72  64  63  64  67  69  70  71  73 ... 93 94 96]
 [ 60  63  67  71  73  65  64  66  68  70  71  72  75 ... 94 95 97]
 ...
[120 118 116 115 113 112 111 110 108 107 108 105 103 ... 66 65 64]
[122 120 118 116 114 113 111 110 109 107 106 104 103 ... 67 66 65]
[123 121 119 117 115 113 112 110 109 108 106 105 104 ... 68 67 66]
[125 122 120 118 115 114 112 111 109 108 107 105 104 ... 69 68 67]
[126 123 121 119 116 114 113 112 110 109 107 106 105 ... 70 69 68]
 ...
 [ 82  84  87  89  91  93  94  95  97  98 100 101 102 ... 36 35 34]
 [ 81  83  86  88  90  92  93  95  96  97  99 100 101 ... 37 36 35]
 [ 80  82  85  87  89  91  92  94  95  96  98  99 100 ... 38 37 36]
 [ 79  81  84  86  88  90  91  93  94  95  97  98  99 ... 39 38 37]
 [ 78  80  83  85  87  89  90  92  93  94  96  97  98 ... 40 39 38]]

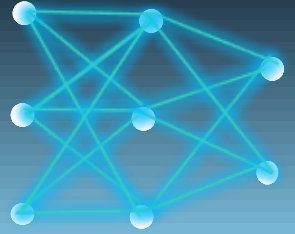
```



[illegible]

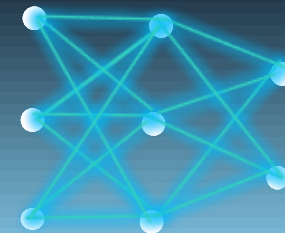


Hierarchical Representations

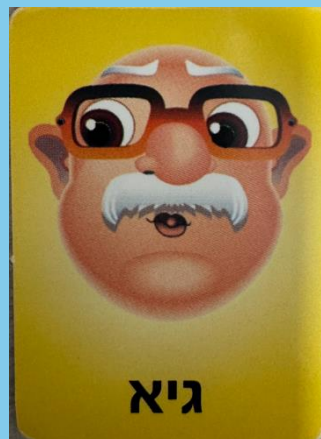


- Deep models are based on the assumption that **complex functions** can be learned as **compositions** of **simpler functions**, enabling hierarchical representation learning through multiple levels of underlying factors
- The relationship between the pixels of an image and a high-level concept such as 'cat' is **highly complex and nonlinear**. However, a **deep neural network** can learn to detect **low-level features**, such as **edges**, in the early layers, and can then combine these in subsequent layers to make **higher-level features** such as **eyes** or **whiskers**, which in turn can be combined in later layers to detect the presence of a **cat**

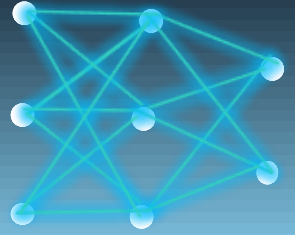
Distributed Representation



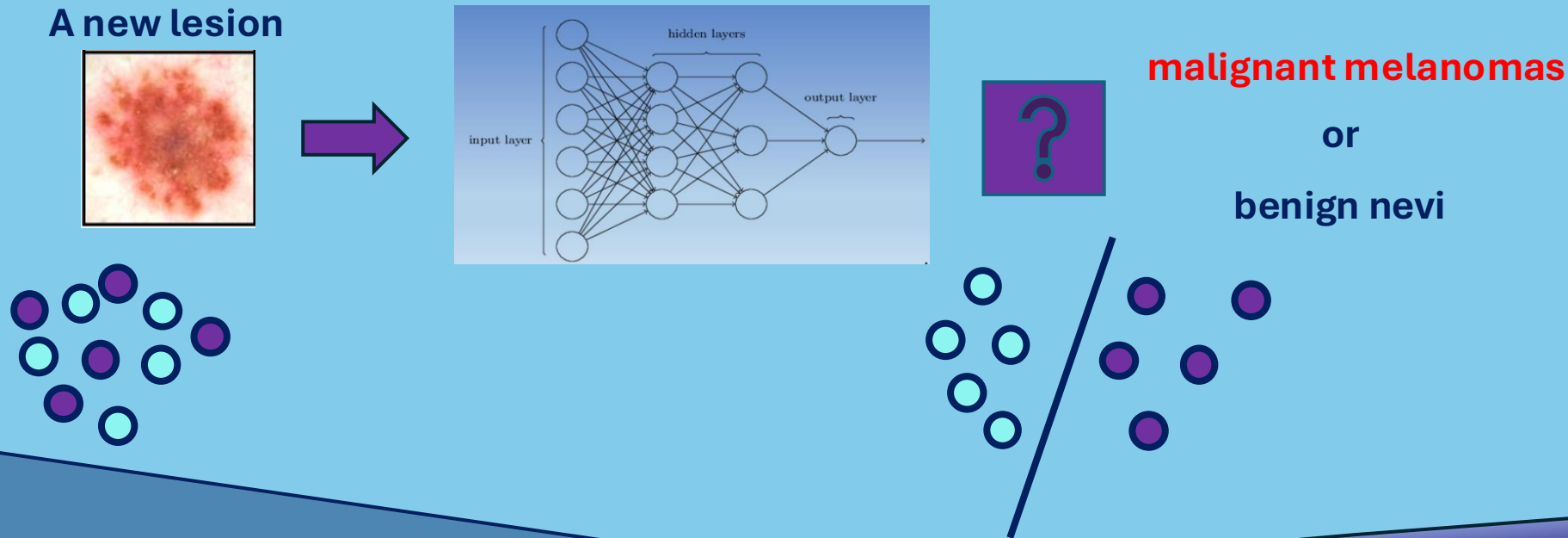
- Neural networks exploit **distributed representations**, where **combinations of hidden units encode features**, enabling a **compact representation** that can grow exponentially in expressiveness



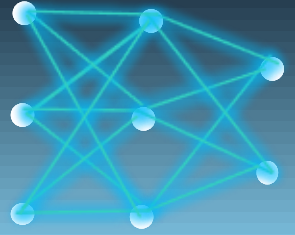
Representation Learning



- Deep neural networks perform **representation learning** by progressively **transforming** data into **new spaces** where tasks become simpler

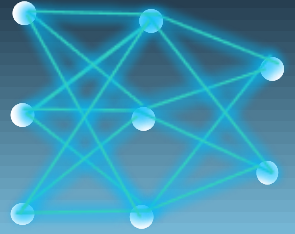


Depth

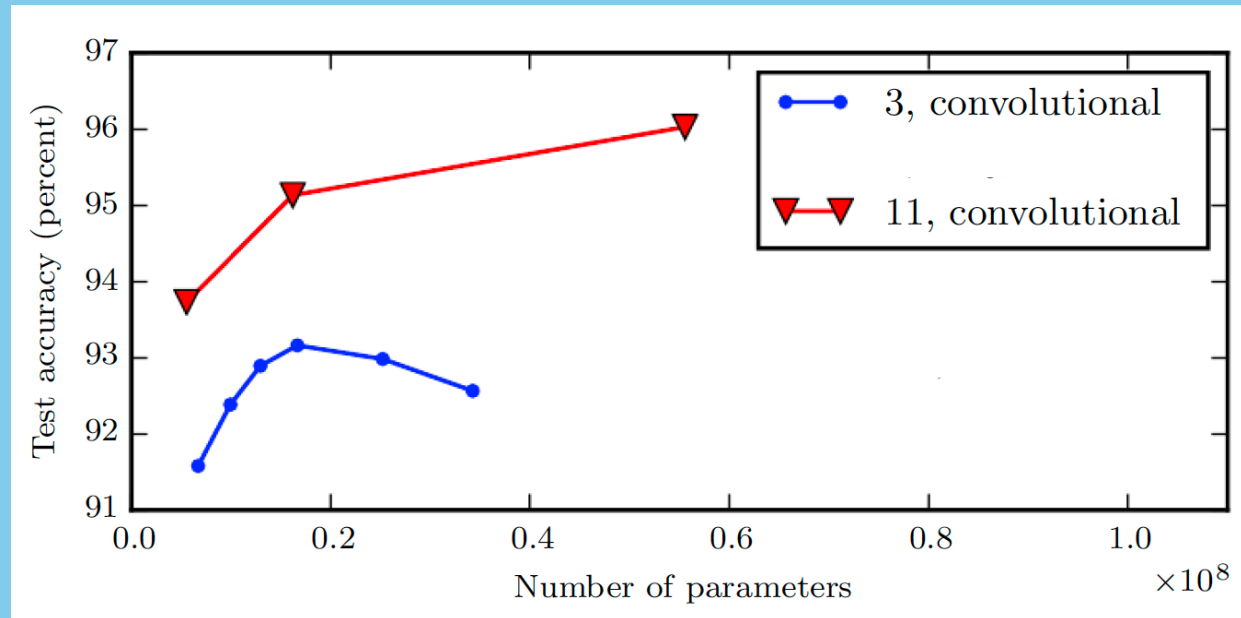
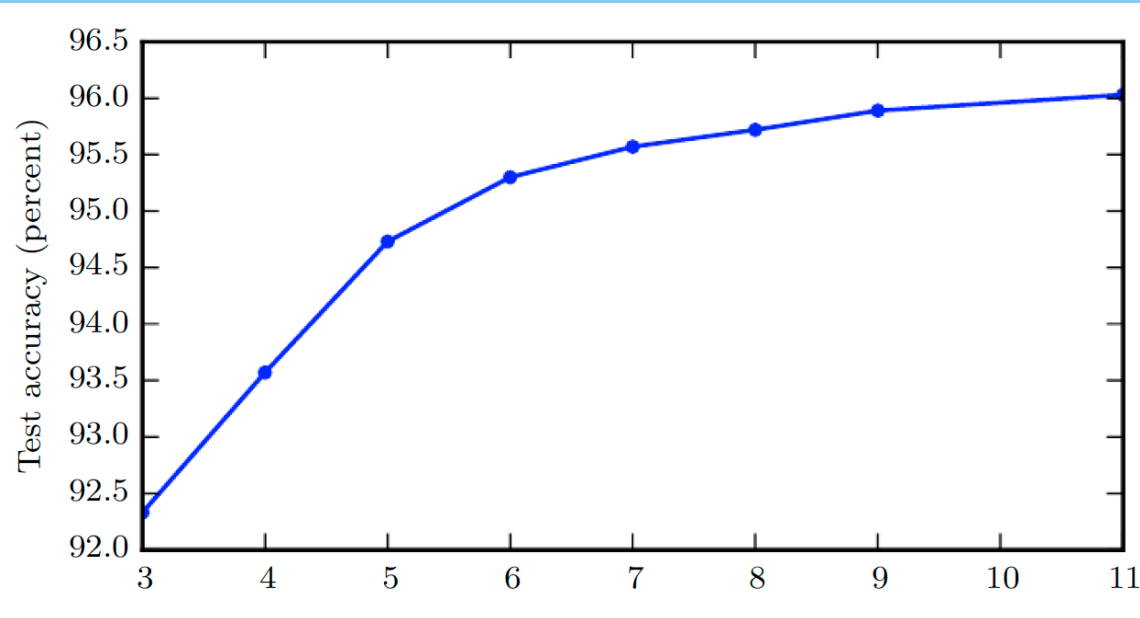


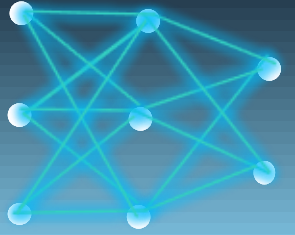
- Although a single-layer feedforward network can represent any function, **deep** architectures often require far **fewer units** and typically achieve **better learning efficiency** and **generalization performance**

Deeper Models Tend to Perform Better

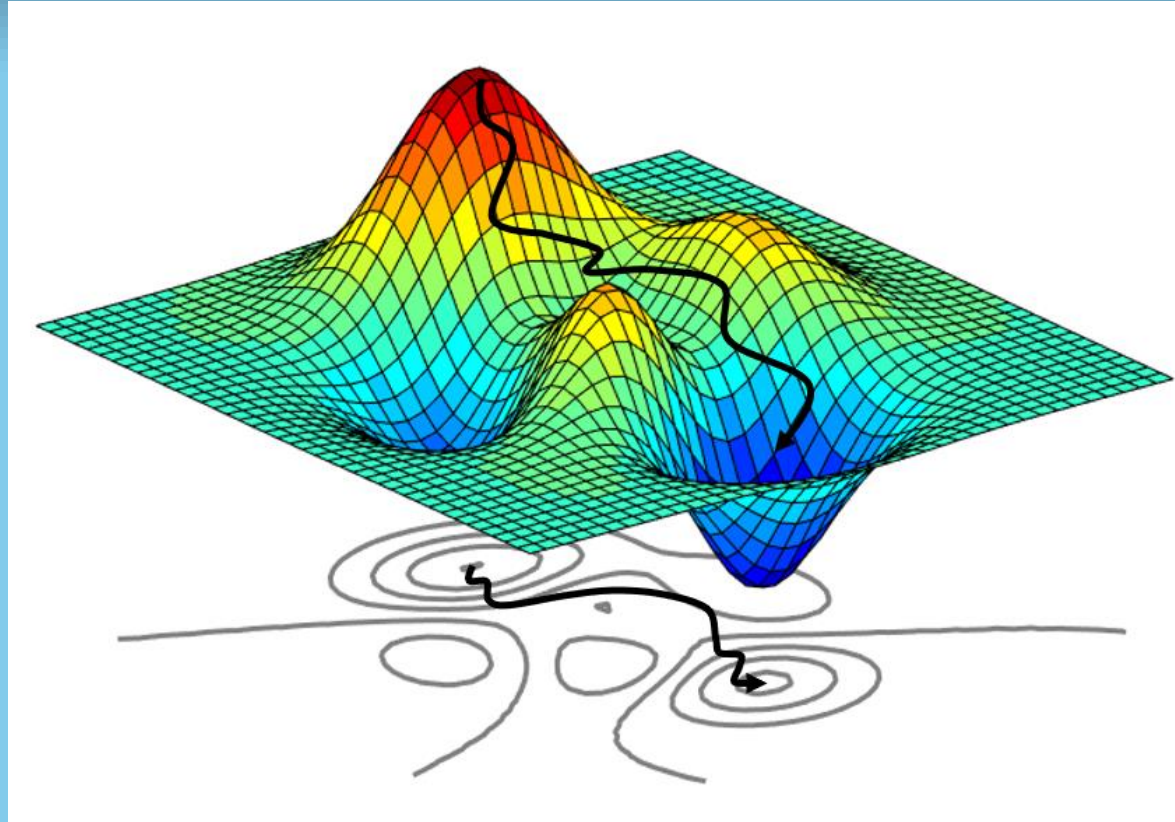


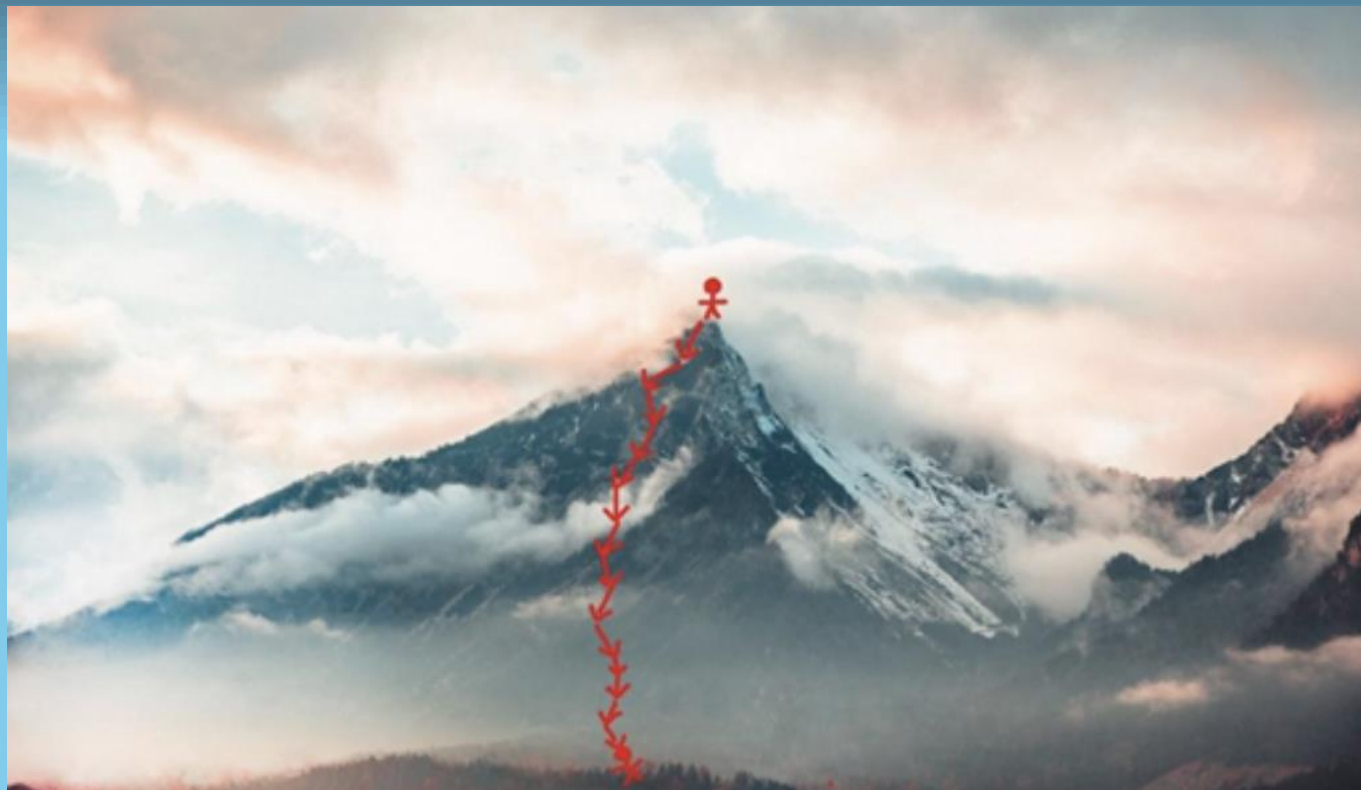
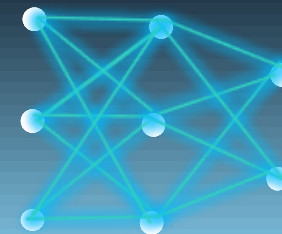
- Empirical evidence shows that **deeper architectures** often **achieve better generalization** across many tasks, suggesting that **depth provides a useful inductive bias** over the functions being learned



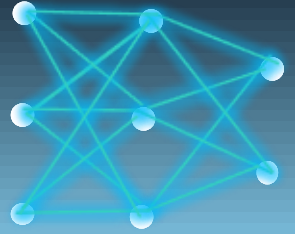


Optimization

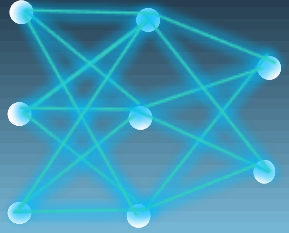




Optimization



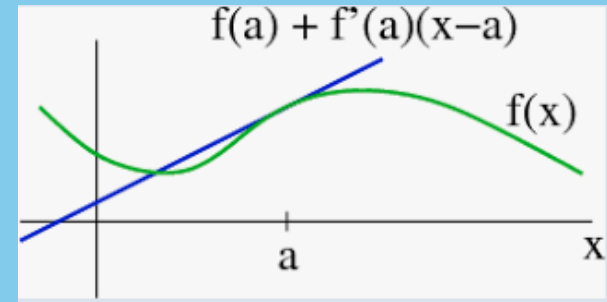
- Most deep learning algorithms involve **optimization**
 - **Optimization** = **minimizing or maximizing** a function $f(x)$ by adjusting x
- The target function is called the **objective function**
- When **minimizing**, it's often called
 - **Cost function**
 - **Loss function**
 - **Error function**

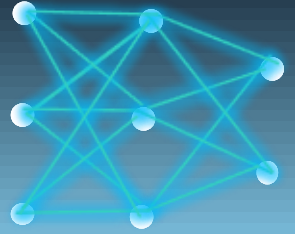


Derivatives and Slope

- Consider a function $y = f(x)$, where x and y are real numbers
- The **derivative**, written as $f'(x)$, gives the **slope** of the function at the point x
- It tells us how a **small change in the input** x affects the **output** y

$$f(x + \varepsilon) \approx f(x) + \varepsilon f'(x)$$



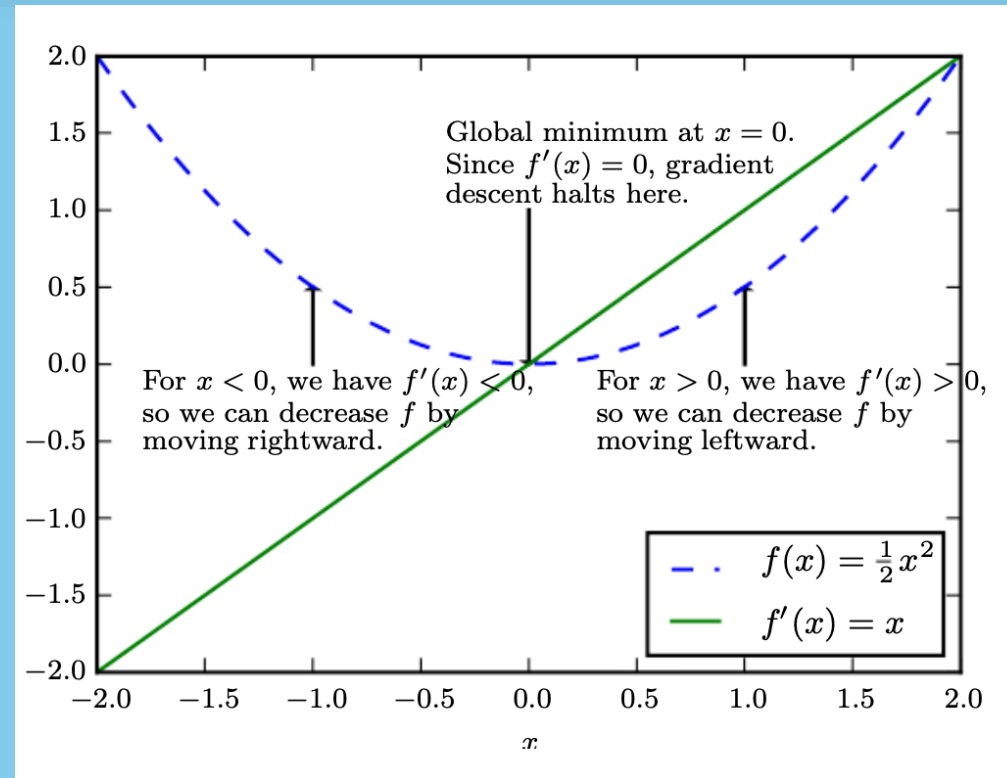
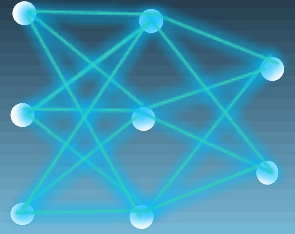


Using Derivatives to Minimize Functions

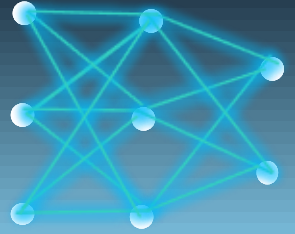
- The **derivative** helps us **minimize a function** by showing how to **adjust** x for a small **improvement** in y
- **Example:**

$$f(x - \varepsilon \operatorname{sign}(f'(x))) < f(x) \text{ for small enough } \varepsilon$$

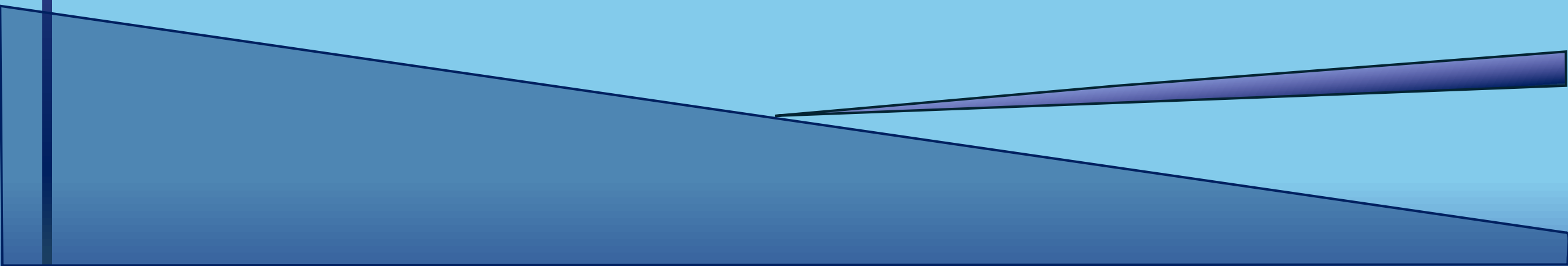
Using Derivatives to Minimize Functions



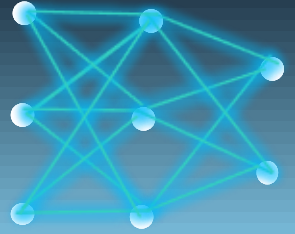
Using Derivatives to Minimize Functions



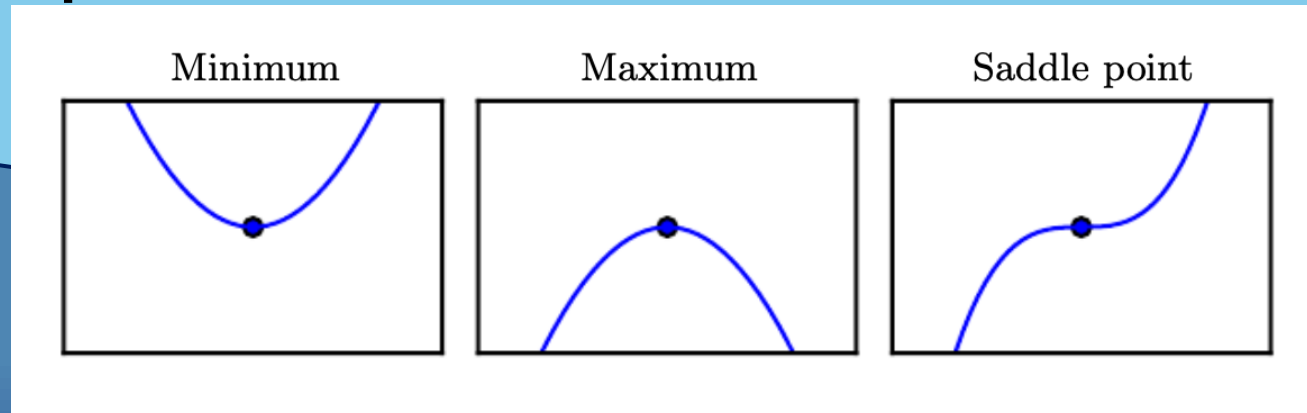
- To **reduce** $f(x)$ move x in **small steps opposite to the derivative's sign**
- This method is called **Gradient-descent**
- When $f'(x) = 0$, we reach a **critical/ stationary point** - **no directional information** is provided



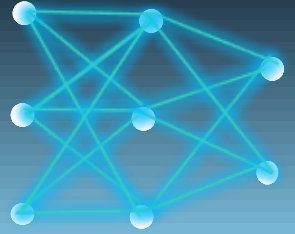
Types of Critical Points



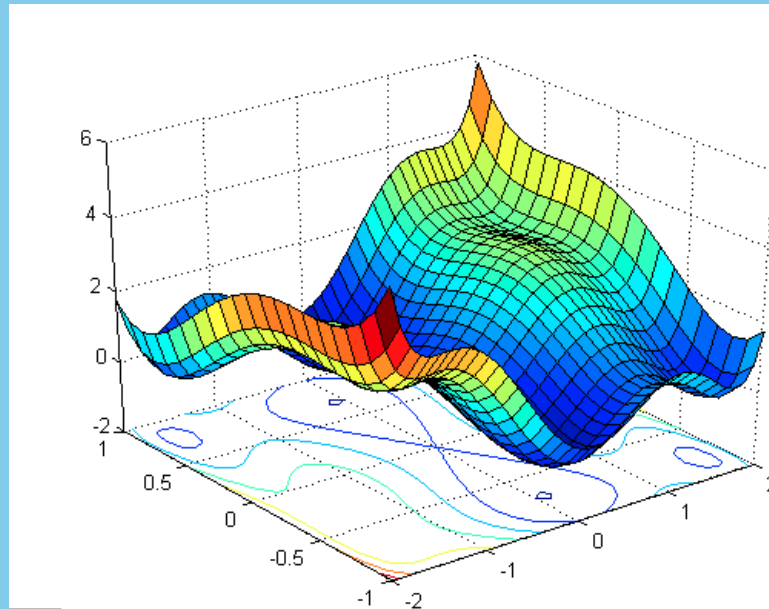
- **A local minimum:**
 - ✓ $f(x)$ is **lower** than at all nearby points
 - ✓ can't decrease further with small steps
- **A local maximum:**
 - ✓ $f(x)$ is **higher** than at all nearby points
 - ✓ Can't increase further with small steps
- **A saddle point:**
 - ✓ A critical point that is **neither a maximum nor a minimum**



Optimization in Deep Networks

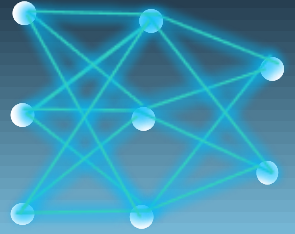


- **Error surfaces** in deep neural networks are often **highly complex**

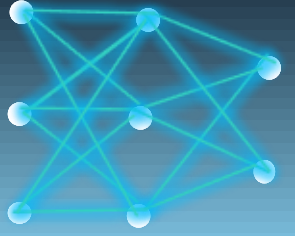


- many local minima
- many local minima
- many saddle points
- flat regions

Role of Initialization in Optimization



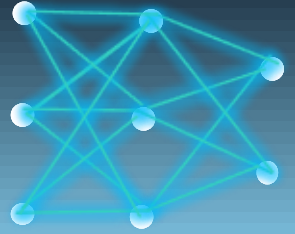
- Due to the **complex error surface**, the solution often **depends** on the initial weights
- To find a sufficiently **good solution**, it may be necessary to:
 - ✓ Run the algorithm **multiple times with different random initialization**
 - ✓ Compare the **performance** on an independent **validation set**



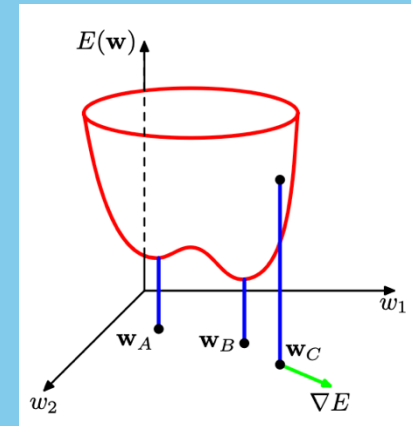
Optimization in Deep Networks

- It was thought that **gradient-based methods** could get stuck in **poor local minima**
- However, in practice, large networks often reach **similar-performance solutions**, under a **variety of initial conditions**

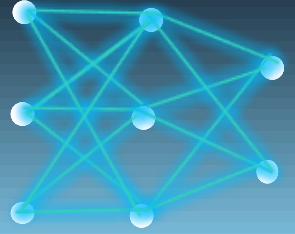
Challenges in Optimization



- Optimization becomes very difficult especially in high-dimensional spaces
- We therefore usually settle for a low value of f that is very low, not necessarily minimal in any formal sense



Functions with Multiple Inputs

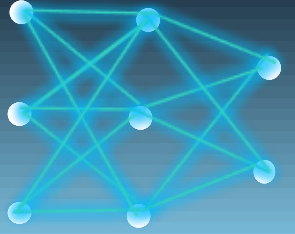


- For functions with **multiple inputs**, we must make use of the concept of **partial derivatives** to understand changes in each variable
- The partial derivative

$$\frac{\partial f(x)}{\partial x_i}$$

measures how f **changes** as only x_i increases

The Gradient



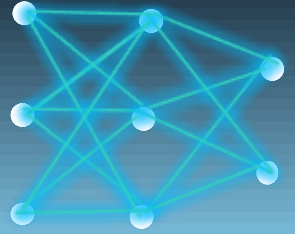
- The **gradient generalizes** the notion of **derivative** for **vector-valued inputs**
- It's a vector of all **partial derivatives**, denoted $\nabla_x f(x)$

$$(\nabla_x f(x))_i = \frac{\partial f(x)}{\partial x_i}$$

- In multiple dimensions, **critical points** occur where:

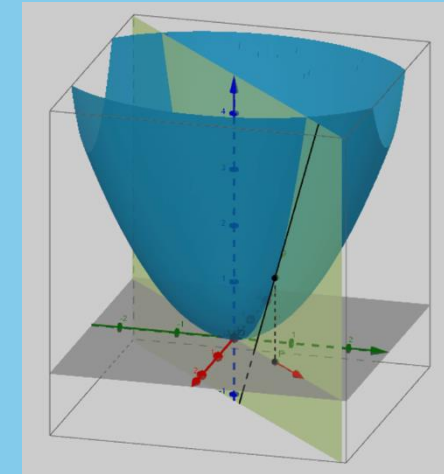
$$\nabla_x f(x) = 0$$

The Directional Derivative



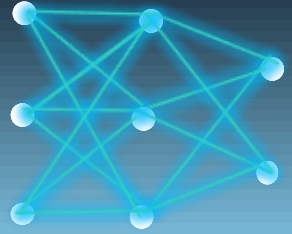
- The **directional derivative** is the slope of f in a given **direction u** ($\|u\|=1$)
- Formally, it's the derivative of $f(x + \alpha u)$ with respect to α , evaluated at $\alpha = 0$

$$\left. \frac{\partial}{\partial \alpha} f(x + \alpha u) \right|_{\alpha=0} = u^T \nabla_x f(x)$$



- It tells how f **changes** as you **move in direction u**

Steepest Descent



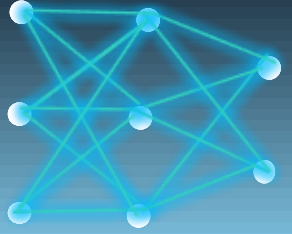
- To **minimize** f , we want the direction of **fastest decrease**

$$\min_{u, u^T u = 1} u^T \nabla_x f(x) = \min_{u, u^T u = 1} \|u\| \|\nabla_x f(x)\| \cos(\theta)$$

- **Minimum occurs when u points opposite the gradient**
- **The gradient points uphill, and the negative gradient points downhill**
- **Moving along the negative gradient is known as **steepest descent** or **gradient descent****



Gradient Descent



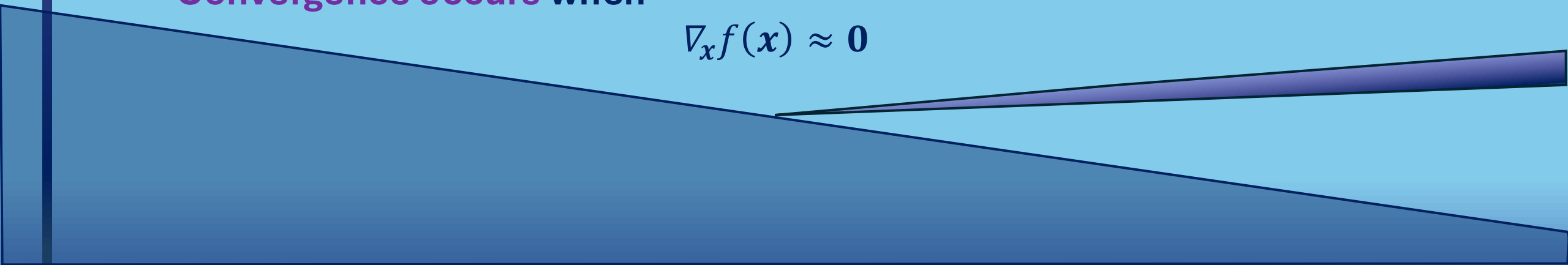
- Steepest descent updates the point as:

$$x' = x - \varepsilon \nabla_x f(x)$$

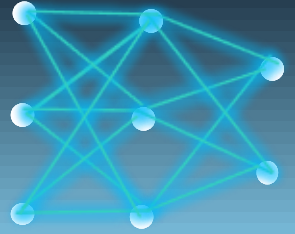
where ε is the **learning rate**: a positive scalar determining **the size of the step**

- Common to use **a small constant** for ε
- **Convergence occurs when**

$$\nabla_x f(x) \approx 0$$



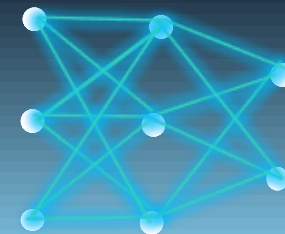
The Hessian Matrix



- For functions with **multiple inputs**, there are **many second derivatives**
- These are organized into the **Hessian matrix**:

$$H(\mathbf{x})_{i,j} = \frac{\partial^2}{\partial x_i \partial x_j} f(\mathbf{x})$$

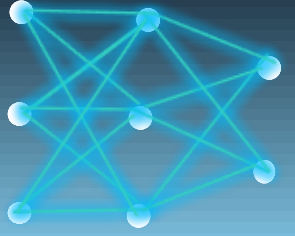
- The Hessian **captures how curvature changes in all directions**



Understanding the Hessian

- The Hessian matrix is **real** and **symmetric**, so it has:
 - ✓ real eigenvalues
 - ✓ An orthonormal basis of eigenvectors

$$H = U\Lambda U^T = \sum_i \lambda_i \mathbf{u}_i \mathbf{u}_i^T$$

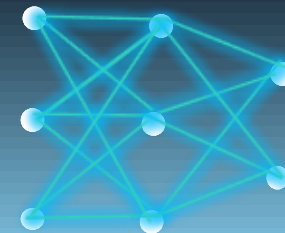


Understanding the Hessian

- The **second derivative in direction d** is

$$\left. \frac{\partial^2}{\partial \alpha^2} f(\mathbf{x} + \alpha \mathbf{d}) \right|_{\alpha=0} = \mathbf{d}^T H(\mathbf{x}) \mathbf{d}$$

- If d is an **eigenvector of H** , the **second derivative** in that direction is given by the **corresponding eigenvalue**



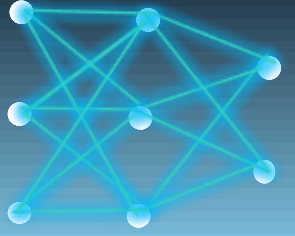
Second-order approximation

- The **second derivative** gives insight into **how effective a gradient descent step will be**
- Use a **second-order Taylor approximation** around point $x^{(0)}$:

$$f(x) \approx f(x^{(0)}) + (x - x^{(0)})^T g + \frac{1}{2} (x - x^{(0)})^T H (x - x^{(0)})$$

✓ **g : gradient at $x^{(0)}$**

✓ **H : Hessian at $x^{(0)}$**



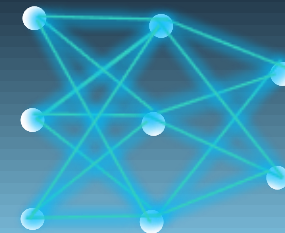
Effect of Gradient Step on $f(\mathbf{x})$

- Using learning rate ε , a gradient descent gives:

$$\mathbf{x} = \mathbf{x}^{(0)} - \varepsilon \mathbf{g}$$

- Substituting into the second-order approximation:

$$f(\mathbf{x}) \approx f(\mathbf{x}^{(0)}) + (\mathbf{x} - \mathbf{x}^{(0)})^T \mathbf{g} + \frac{1}{2} (\mathbf{x} - \mathbf{x}^{(0)})^T \mathbf{H} (\mathbf{x} - \mathbf{x}^{(0)})$$



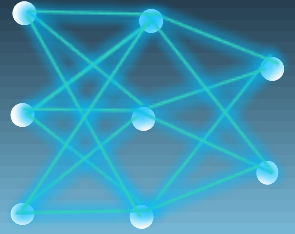
Effect of Gradient Step on $f(\mathbf{x})$

$$\begin{aligned}\mathbf{x} &= \mathbf{x}^{(0)} - \varepsilon \mathbf{g} \\ f(\mathbf{x}) &\approx f(\mathbf{x}^{(0)}) + (\mathbf{x} - \mathbf{x}^{(0)})^T \mathbf{g} + \frac{1}{2} (\mathbf{x} - \mathbf{x}^{(0)})^T \mathbf{H} (\mathbf{x} - \mathbf{x}^{(0)}) \\ f(\mathbf{x}^{(0)} - \varepsilon \mathbf{g}) &\approx \boxed{f(\mathbf{x}^{(0)})} - \boxed{\varepsilon \mathbf{g}^T \mathbf{g}} + \boxed{\frac{1}{2} \varepsilon^2 (\mathbf{g}^T \mathbf{H} \mathbf{g})}\end{aligned}$$

- **Three components:**

- ✓ **Original value** of the function
- ✓ **Expected decrease** due to the gradient (slope)
- ✓ **Correction** for the **curvature** (via Hessian)

Step-size and Curvature



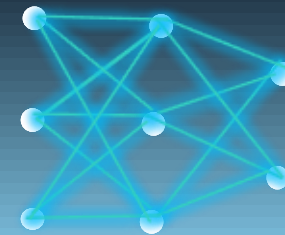
$$f(\mathbf{x}^{(0)} - \varepsilon \mathbf{g}) \approx f(\mathbf{x}^{(0)}) - \varepsilon \mathbf{g}^T \mathbf{g} + \frac{1}{2} \varepsilon^2 (\mathbf{g}^T \mathbf{H} \mathbf{g})$$

- If the **curvature** term $\mathbf{g}^T \mathbf{H} \mathbf{g}$ is **too large**, the gradient descent step can actually increase $f(\mathbf{x})$ (move uphill)
- To **minimize** the second order approximation (when $\mathbf{g}^T \mathbf{H} \mathbf{g} > 0$), the optimal step-size is:

$$\varepsilon^* = \frac{\mathbf{g}^T \mathbf{g}}{\mathbf{g}^T \mathbf{H} \mathbf{g}}$$

Worst case?

Quadratic Approximation at a Critical Point



- At a **critical point**, the **gradient is zero**:

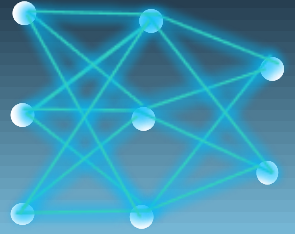
$$\nabla_x f(\mathbf{x}) \Big|_{\mathbf{x}^*} = \mathbf{0}$$

- The quadratic approximation simplifies to:

$$f(\mathbf{x}) \approx f(\mathbf{x}^*) + \frac{1}{2} (\mathbf{x} - \mathbf{x}^*)^T H (\mathbf{x} - \mathbf{x}^*)$$

- **No linear term** since there is **no slope at a critical point**. Behavior near a critical point is entirely governed by the **Hessian** (curvature)

Eigen-decomposition of the Hessian



- The **eigenvalue equation** for the **Hessian**:

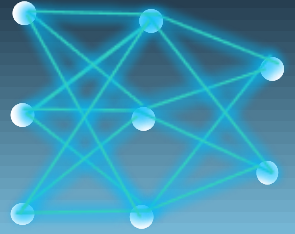
$$H\mathbf{u}_i = \lambda_i\mathbf{u}_i$$

- **Orthonormality** condition:

$$\mathbf{u}_i^T \mathbf{u}_j = \delta_{ij}$$

- The eigenvectors $\{\mathbf{u}_i\}$ form a **complete orthonormal basis**

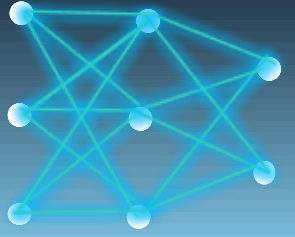
Eigen-decomposition of the Hessian



- We can express:

$$\mathbf{x} - \mathbf{x}^* = \sum_i \alpha_i \mathbf{u}_i$$

where $\mathbf{x}^*$: a critical point and α_i : coefficients along each eigenvector



Analyzing Curvature via Eigenvalues

- Quadratic approximation around a critical point:

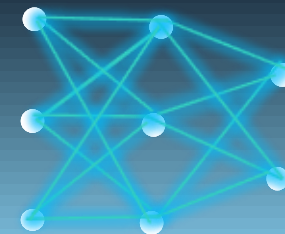
$$f(\mathbf{x}) \approx f(\mathbf{x}^*) + \frac{1}{2} (\mathbf{x} - \mathbf{x}^*)^T H (\mathbf{x} - \mathbf{x}^*)$$

- Expand displacement in eigen-basis:

$$\mathbf{x} - \mathbf{x}^* = \sum_i \alpha_i \mathbf{u}_i$$

- Substituting yields:

$$f(\mathbf{x}) \approx f(\mathbf{x}^*) + \frac{1}{2} \sum_i \lambda_i \alpha_i^2$$



Analyzing Curvature via Eigenvalues

- Expand displacement in eigen-basis:

$$\mathbf{x} - \mathbf{x}^* = \sum_i \alpha_i \mathbf{u}_i$$

- Substituting yields:

$$f(\mathbf{x}) \approx f(\mathbf{x}^*) + \frac{1}{2} \sum_i \lambda_i \alpha_i^2$$

- Varying only one α_j ($\alpha_i = 0$ for $i \neq j$):

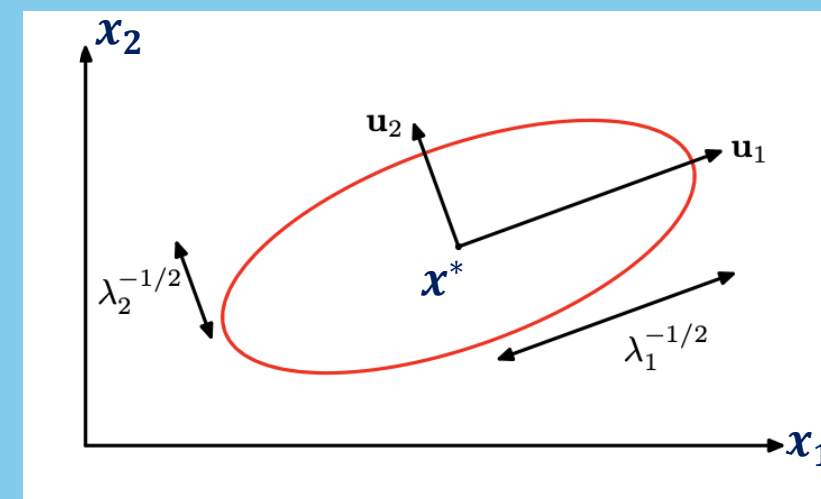
- ✓ If $\lambda_j > 0 \rightarrow$ function **increases**
- ✓ If $\lambda_j < 0 \rightarrow$ function **decreases**

Conclusion

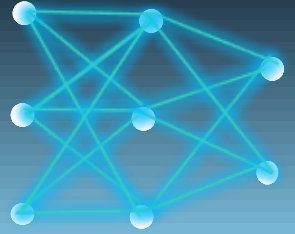
All $\lambda_i > 0 \rightarrow$ Local **minimum**

All $\lambda_i < 0 \rightarrow$ Local **maximum**

Mixed signs $\rightarrow$ **Saddle point**



Convergence Analysis



- The quadratic approximation around a critical point simplifies to:

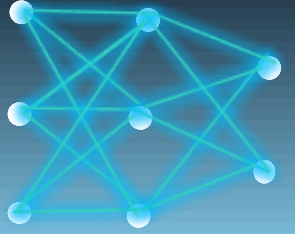
$$f(\mathbf{x}) \approx f(\mathbf{x}^*) + \frac{1}{2} (\mathbf{x} - \mathbf{x}^*)^T H (\mathbf{x} - \mathbf{x}^*)$$

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = H (\mathbf{x} - \mathbf{x}^*)$$

$$\mathbf{x} - \mathbf{x}^* = \sum_i \alpha_i \mathbf{u}_i$$

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = H \sum_i \alpha_i \mathbf{u}_i = \sum_i \alpha_i H \mathbf{u}_i = \sum_i \alpha_i \lambda_i \mathbf{u}_i$$

Convergence Analysis

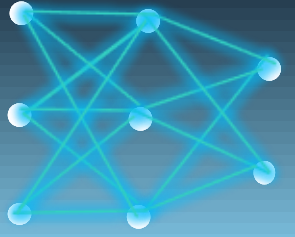


$$\mathbf{x}^{(new)} - \mathbf{x}^* = \sum_i \alpha_i^{(new)} \mathbf{u}_i$$

$$\mathbf{x}^{(old)} - \mathbf{x}^* = \sum_i \alpha_i^{(old)} \mathbf{u}_i$$

$$\mathbf{x}^{(new)} - \mathbf{x}^{(old)} = \sum_i \left(\alpha_i^{(new)} - \alpha_i^{(old)} \right) \mathbf{u}_i$$

$$\nabla_x f(\mathbf{x}) \Big|_{\mathbf{x}^{(old)}} = H \left(\mathbf{x}^{(old)} - \mathbf{x}^* \right) = H \sum_i \alpha_i^{(old)} \mathbf{u}_i = \sum_i \alpha_i^{(old)} \lambda_i \mathbf{u}_i$$



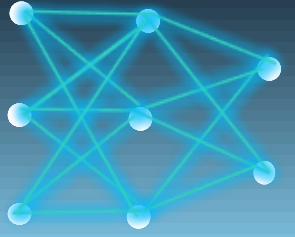
Convergence Analysis

$$\begin{aligned}\mathbf{x}^{(new)} &= \mathbf{x}^{(old)} - \varepsilon \nabla_{\mathbf{x}} f(\mathbf{x}) \Big|_{\mathbf{x}^{(old)}} \\ \mathbf{x}^{(new)} - \mathbf{x}^{(old)} &= -\varepsilon \nabla_{\mathbf{x}} f(\mathbf{x}) \Big|_{\mathbf{x}^{(old)}}\end{aligned}$$

$$\mathbf{x}^{(new)} - \mathbf{x}^{(old)} = \sum_i (\alpha_i^{(new)} - \alpha_i^{(old)}) \mathbf{u}_i$$

$$\nabla_{\mathbf{x}} f(\mathbf{x}) \Big|_{\mathbf{x}^{(old)}} = \sum_i \alpha_i^{(old)} \lambda_i \mathbf{u}_i$$

$$\sum_i (\alpha_i^{(new)} - \alpha_i^{(old)}) \mathbf{u}_i = -\varepsilon \sum_i \alpha_i^{(old)} \lambda_i \mathbf{u}_i$$



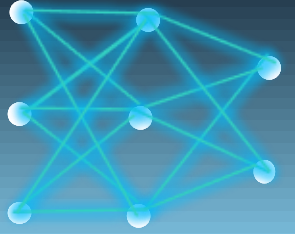
Convergence Analysis

$$\sum_i (\alpha_i^{(new)} - \alpha_i^{(old)}) \mathbf{u}_i = -\varepsilon \sum_i \alpha_i^{(old)} \lambda_i \mathbf{u}_i$$

$$(\alpha_i^{(new)} - \alpha_i^{(old)}) = -\varepsilon \alpha_i^{(old)} \lambda_i$$

$$\alpha_i^{(new)} = (1 - \varepsilon \lambda_i) \alpha_i^{(old)}$$

Convergence Analysis

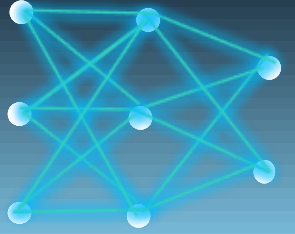


$$\alpha_i^{(new)} = (1 - \varepsilon \lambda_i) \alpha_i^{(old)}$$

$$\alpha_i^{(T)} = (1 - \varepsilon \lambda_i)^T \alpha_i^{(0)}$$

$$\alpha_i = \mathbf{u}_i^T (\mathbf{x} - \mathbf{x}^*)$$

Convergence Analysis



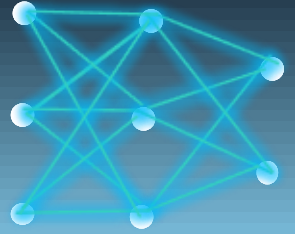
$$\alpha_i^{(T)} = (1 - \varepsilon \lambda_i)^T \alpha_i^{(0)}$$

$$|1 - \varepsilon \lambda_i| < 1$$

$$0 < \varepsilon < \frac{2}{\lambda_{\max}}$$

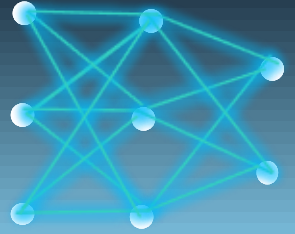
Optimal ε ?

Poor Conditioning Hurts Gradient Descent

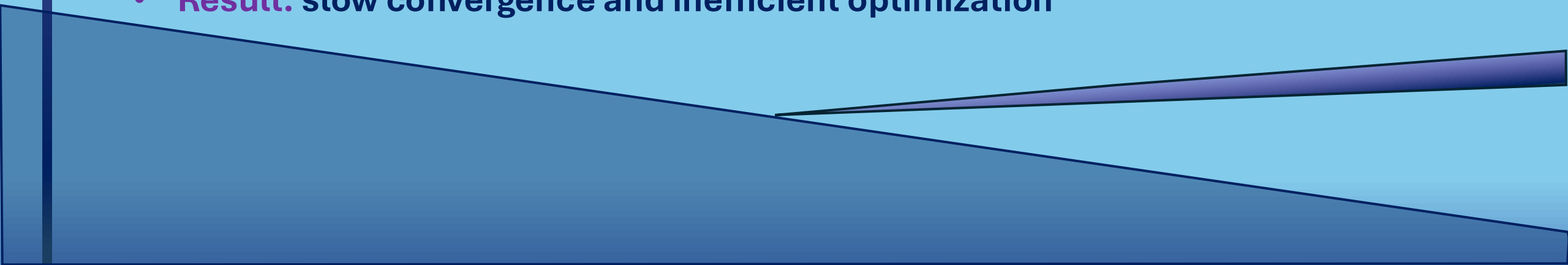


- A **poorly conditioned Hessian** has a large condition number (ratio of largest to smallest eigenvalue)
- This causes **uneven gradients**:
 - ✓ One direction: steep → **fast change**
 - ✓ Another: flat → **slow change**
- Gradient descent struggles because:
 - ✓ It uses the **same step-size** in **all directions**
 - ✓ It may **zigzag in steep directions** and **barely move in flatter ones**
- As a result, it **takes many steps** to reach a good solution

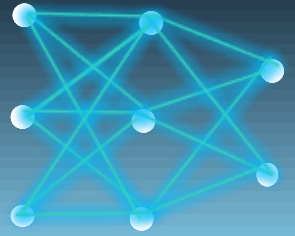
Step-size Trade-off in Poorly Conditioned Problems



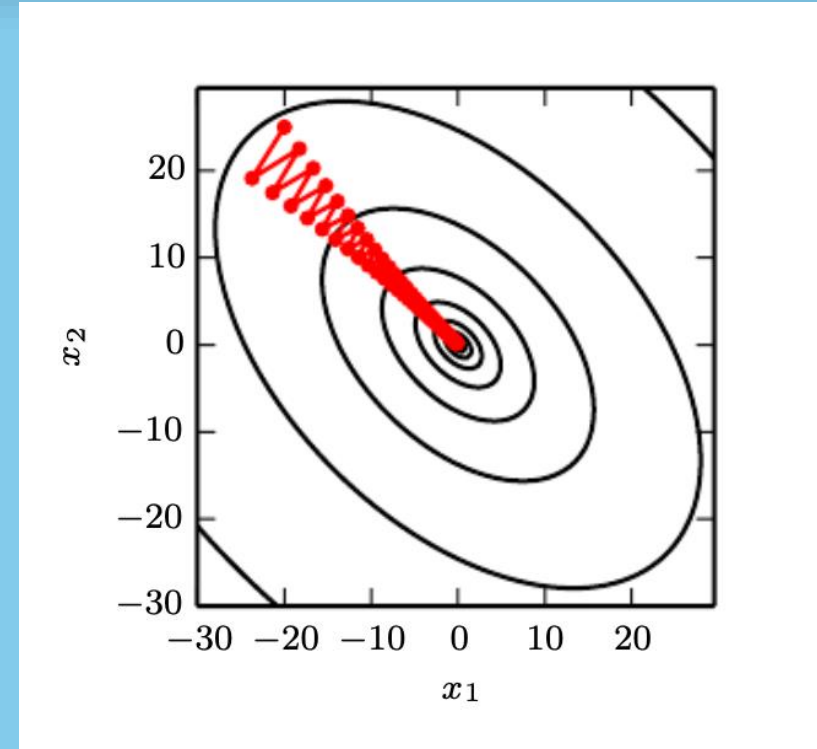
- Hard **to choose** a single **good step size** when curvature varies by direction
- Step **size** must be **small enough** to **avoid overshooting** in directions with **strong positive curvature**
- But then it's **too small** to make progress in **other directions with less curvature**
- **Result:** slow convergence and inefficient optimization

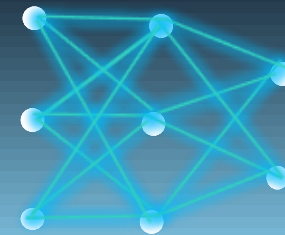


Gradient Descent to minimize a quadratic function



- Hessian matrix has condition number 5
- Gradient descent wastes steps going up and down steep canyon walls
- step size is too large for narrow directions, so it:
 - ✓ Overshoots the bottom
 - ✓ Then bounces back on the next step
- This results in a zigzag path and slow progress towards the minimum





Thank you